



React

# Fundamentals of React

## What is React?

- React is a JavaScript library to build web applications
- Designed to be lightweight and simple to understand
- Not a framework: React is focused on rendering HTML. Additional features require third-party libraries

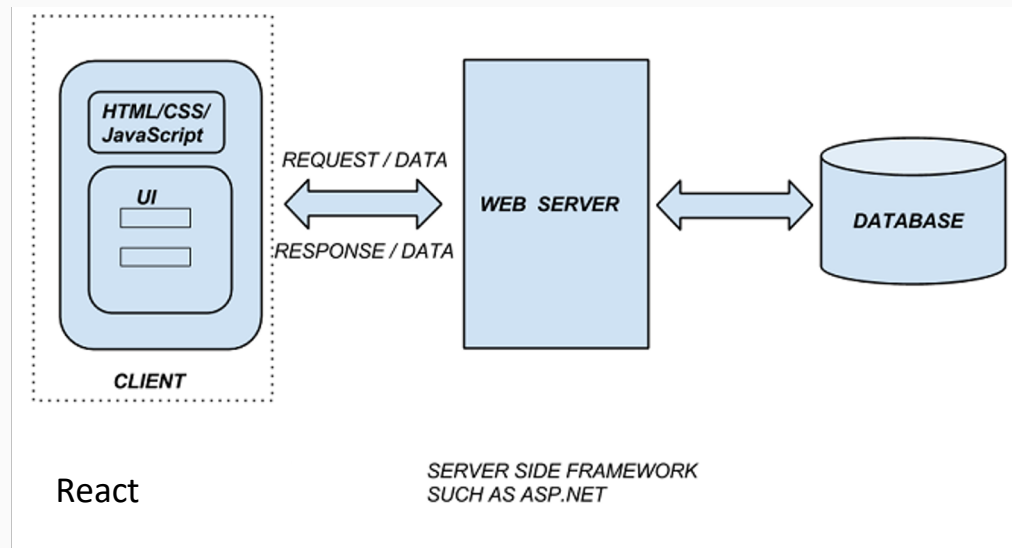
## What is React?

- **React** lets you compose complex UIs from small and isolated pieces of code called “components”
- When our data changes, **React** will efficiently update and re-render our components. It “**reacts**” to data updates
- Creating components is equivalent to creating new, custom HTML elements that can be used in multiple different places within an app (think tabs, date pickers, forms, etc.)

## How is it different from PHP, ASP, JSP?

With React, the front-end code is independent from the back-end

The web server becomes a web-service that typically outputs JSON data, not dynamic HTML or CSS



## Creating a New React App

- A **build tool** is required to run source code, provide a development server, and to package an app for deployment to a production server (**Vite**, Parcel, or Rsbuild)

## Lab 1: Create a New React App



- Create a new React app by using **create-vite**
- Open the app's code in **Visual Studio Code**
- Use the **development server** to test the app

# Introducing JSX

Consider this variable declaration:

```
const element = <h1>Hello, world!</h1>;
```

- It is called **JSX**, and it is a syntax extension to JavaScript. We recommend using it with React to describe what the UI should look like.
- **JSX** may remind you of a template language, but it comes with the full power of JavaScript.
- **JSX** produces React “elements”.

JSX is turned into  
DOM commands

Babel compiles JSX down to  
`React.createElement()`  
calls.

These two examples are  
identical:

```
const element = (  
  <h1 className="greeting">  
    Hello, world!  
  </h1>  
);
```

```
const element = React.createElement(  
  'h1',  
  {className: 'greeting'},  
  'Hello, world!'  
);
```



## Why JSX?

- React doesn't require using JSX, but most people find it helpful as a visual aid when working with UI markup inside of JavaScript code
- Here is how we can ask React to render a JSX element in the DOM:

```
const root = createRoot(document.getElementById('root'));  
root.render(<h1>Hello, world!</h1>);
```

## Embedding Expressions in JSX

In this example, we declare a variable called `name` and use it inside JSX by wrapping it in curly braces

You can put any valid JavaScript expression inside the curly braces in JSX.

For example, `2 + 2`, `user.firstName`, or `formatName(user)` are all valid JSX expressions.

```
const name = "New React Dev";

const root = createRoot(
  document.getElementById("app")
);

root.render(<h1>Hello, {name}</h1>);
```

## JSX is Javascript

After compilation, JSX expressions become regular JavaScript function calls and evaluate to JavaScript objects.

This means that you can use JSX inside of `if` statements and `for` loops, assign it to variables, accept it as arguments, and return it from functions.

```
function getGreeting(user) {  
  if (user) {  
    return <h1>Hello, {formatName(user)}!</h1>;  
  }  
  return <h1>Hello, Stranger.</h1>;  
}
```

## Specifying Attributes with JSX

You may use quotes to specify string literals as attributes:

```
const element = <div tabIndex="0"></div>;
```

You may also use curly braces to embed a JavaScript expression in an attribute:

```
const element = <img src={user.avatarUrl}></img>;
```

## Creating sibling elements with JSX

Sometimes, we want to return sibling elements with no defined parent:

```
<td>Hello</td>  
<td>World</td>
```

In that case, we can wrap those elements in a **fragment element** since a JSX expression must represent a single element:

```
<React.Fragment>  
  <td>Hello</td>  
  <td>World</td>  
</React.Fragment>
```

OR

```
<>  
  <td>Hello</td>  
  <td>World</td>  
</>
```

## What are Elements?

- **Elements** are the smallest building blocks of React apps.
- Unlike browser DOM elements, React elements are plain objects, and are cheap to create.
- React DOM takes care of updating the DOM to match the React elements.

An element describes what you want to see on the screen:

```
const element = <h1>Hello, world</h1>;
```

## Rendering an Element into the DOM

We need a `<div>` somewhere in our main index HTML file:

```
<div id="root"></div>
```

We call this a “root” DOM node because everything inside it will be managed by React DOM.

Applications built with just React usually have a single root DOM node in their **index.html**

If you are integrating React into an existing app, you may have as many isolated root DOM nodes as you like.

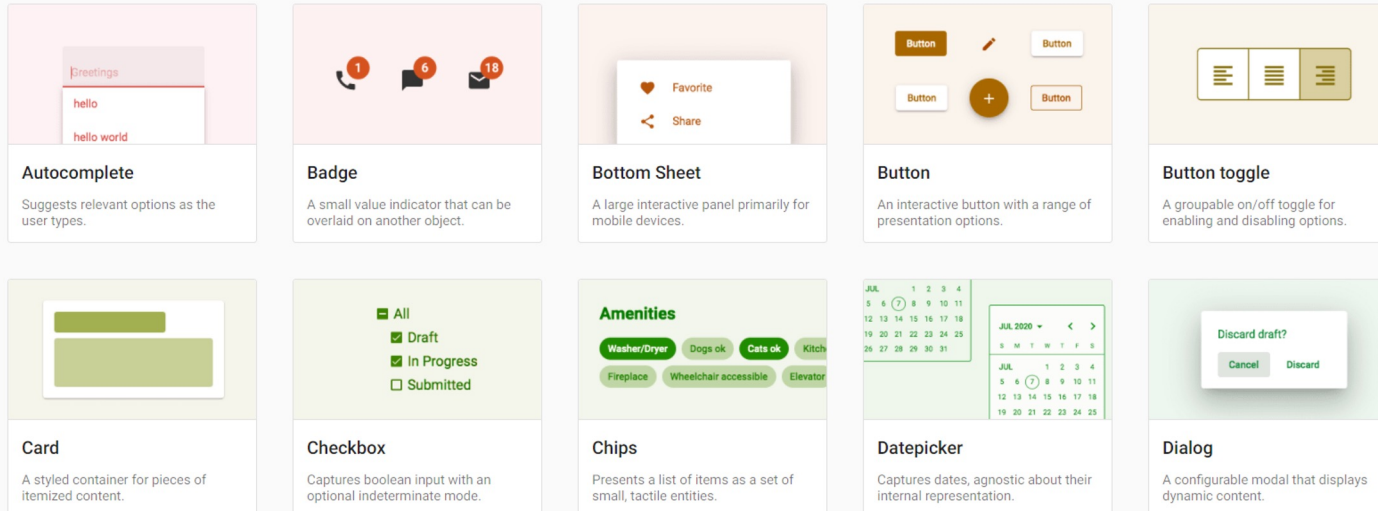
## Virtual DOM defined

- React uses a virtual, in-memory representation of the DOM tree, known as the “**virtual DOM**”
- When a component re-renders, **React is creating a diff** of the virtual DOM **before and after** re-rendering.
- If the diff has any changes to apply to the DOM then React makes these **incremental changes** in the browser
- As a result, only the portions of the UI that need to be re-rendered actually get re-rendered



# What are Components?

- Components let you split the user interface into independent, reusable pieces (think of them as widgets or custom HTML elements).
- A React app is made of several components.
- We can create our own components or build upon components from third-party libraries (e.g., <https://material-ui.com>):



## Function and Class Components

- The simplest way to define a component is to write a function:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

- This function is a valid React component because it accepts a single “props” (which stands for properties) object and returns a React element
- We call such components “**function components**” because they are literally JavaScript functions

## Rendering a Component

Previously, we only encountered React elements that represent DOM tags:

```
const element = <div />;
```

However, elements can also represent user-defined components:

```
const element = <Welcome name="Sara" />;
```

# Rendering a Component

Let's recap what happens in this example:

1. We call `render()` with the `<Welcome name="Sara" />` element.
2. React calls the `Welcome` component with `{name: 'Sara'}` as the props.
3. Our `Welcome` component returns a `<h1>Hello, Sara</h1>` element as the result.
4. React DOM efficiently updates the DOM to match `<h1>Hello, Sara</h1>`

For example, this code renders “Hello, Sara” on the page:

```
// This is a component with props
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}

const element = <Welcome name="Sara" />;
root.render(element);
```

# Composing Components

Components can refer to other components in their output.

This lets us use the same component abstraction for any level of detail.

A button, a form, a dialog, a screen: in React apps, all those are commonly expressed as components.

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}  
  
function App() {  
  return (  
    <div>  
      <Welcome name="Sara" />  
      <Welcome name="Cahal" />  
      <Welcome name="Edite" />  
    </div>  
  );  
}
```

## Handling Events

For example, the HTML:

```
<button onclick="activateLasers()">  
  Activate Lasers  
</button>
```

is slightly different in React:

```
<button onClick={activateLasers}>  
  Activate Lasers  
</button>
```

## Handling Events: Function Component

With function components, event handlers are usually other functions defined outside the scope of the current component.

```
function handleClick() {  
  console.log('clicked');  
}  
  
function Button() {  
  return <button onClick={handleClick}>Click Me!</button>;  
}  
  
const element = <Button />;  
ReactDOM.render(element, document.getElementById('root'));
```

## Conditional Rendering

- Conditional rendering in React works the same way conditions work in JavaScript.
- We can use `if` or the inline conditional operators to decide whether to display an element or not:

```
<div>  
  The user is <b>{isLoggedIn ? 'currently' : 'not'}</b> logged in.  
</div>
```

- In this example, the expression will render “currently”, when `isLoggedIn` is `true`, “not” otherwise



## Conditional Rendering

Let's consider these two function components:

```
function UserGreeting(props) {  
  return <h1>Welcome back!</h1>;  
}  
  
function GuestGreeting(props) {  
  return <h1>Please sign up.</h1>;  
}
```

## Conditional Rendering

We'll create a `Greeting` component that displays either of these components depending on whether a user is logged in.

This example renders a different greeting depending on the value of `isLoggedIn` prop.

```
function Greeting(props) {  
  const isLoggedIn = props.isLoggedIn;  
  if (isLoggedIn) {  
    return <UserGreeting />;  
  }  
  return <GuestGreeting />;  
}
```

## Element Variables

You can use variables to store elements.

This can help you conditionally render a part of the component while the rest of the output doesn't change.

```
render() {  
  const isLoggedIn = this.state.isLoggedIn;  
  let button;  
  if (isLoggedIn) {  
    button = <LogoutButton onClick={this.handleLogoutClick} />;  
  } else {  
    button = <LoginButton onClick={this.handleLoginClick} />;  
  }  
  
  return (  
    <div>  
      <Greeting isLoggedIn={isLoggedIn} />  
      {button}  
    </div>  
  );  
}
```

# useState() for function components

**useState** returns the current state value and a function to update that value - in an array.

It takes a default value as a parameter.

```
import React, { useState } from 'react';

function Example() {
  // Declare a new state variable, which we'll call "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

## useState() Hook

You can use the useState Hook more than once in a single component

Note the **array destructuring syntax** that lets us give different names to the state variables we received when calling **useState**.

```
import { useState } from 'react';

function MyComponent() {
  const [age, setAge] = useState(28);
  const [name, setName] = useState('Taylor');
  const [todos, setTodos] = useState(() => createTodos());
  // ...
}
```

## useState() Hook

If the new state is computed based on the previous state, we can pass a function to the setter to compute that new value as follows:

```
const [count, setCount] = React.useState(0);

return (
  <div>
    <p>You clicked {count} times</p>
    <button onClick={() => setCount(c => c + 1)}>
```

## Using State Correctly

### Do not Modify State Directly

For example, this will not re-render a component:

```
// Wrong  
this.state.comment = 'Hello';
```

Instead, use **setState()**

```
// Correct  
this.setState({comment: 'Hello'});
```

## With React, Data Flows Down

This is commonly called a “top-down” or “unidirectional” data flow. Any state is always owned by **some specific component**, and any data or UI derived from that state can only affect components “**below**” them in the tree.

If you imagine a component tree as a **waterfall** of props, each component’s state is like an additional water source that joins it at an arbitrary point but also **flows down**.



## Data Flows Down

A component may choose to pass its state down as props to its child components:

```
<FormattedDate date={this.state.date} />
```

The **FormattedDate** component would receive the date in its props and wouldn't know whether it came from the Clock's state, from the Clock's props, or was typed by hand:

```
function FormattedDate(props) {  
  return <h2>It is {props.date.toLocaleTimeString()}.</h2>;  
}
```

# useEffect() Hook

**useEffect** is the equivalent of lifecycle methods for function components.

It takes a function as a parameter.

By using this Hook, you tell React that your component needs to do something after **each render**.

```
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  // Similar to componentDidMount and componentDidUpdate:
  useEffect(() => {
    // Update the document title using the browser API
    document.title = `You clicked ${count} times`;
  });
}
```

## useEffect() Hook with empty array

**useEffect** takes a second parameter that determines when the effect should run.

Using an **empty array** means that we want the effect to run **just once** after the initial render (equivalent to the **componentDidMount** lifecycle method for class components)

```
let [todos, setTodos] = React.useState([]);

React.useEffect(() => {
  fetch('https://jsonplaceholder.typicode.com/todos?userId=1')
    .then(response => response.json())
    .then(json => setTodos(json))
}, []);
```

## Cleaning up before unmounting

**useEffect** also accepts a returned function that will run before unmounting a component.

This is a perfect solution to unsubscribe/clear some timers as we would do with **componentWillUnmount** in a class component.

```
function FriendStatus(props) {  
  // ...  
  useEffect(() => {  
    // ...  
    ChatAPI.subscribeToFriendStatus(props.friend.id, handleStatusChange);  
    return () => {  
      ChatAPI.unsubscribeFromFriendStatus(props.friend.id, handleStatusChange);  
    };  
  });  
}
```

## useEffect() Hook with non-empty array

The second parameter of **useEffect** is an **array of dependencies** that need to change values for the effect to actually run.

This is a perfect solution when you have multiple side-effects based on different props/state values.

```
useEffect(() => {  
  document.title = `You clicked ${count} times`;  
}, [count]); // Only re-run the effect if count changes
```

## Using fetch()

- The **Fetch API** is standard in modern browsers and does not require an additional install
- **fetch()** returns a promise that resolves to a **Response** object

```
fetch('http://localhost:8000/data').then(response => response.json())
```

- The interface for the **Response** object can be found here:  
<https://developer.mozilla.org/en-US/docs/Web/API/Response>
- In our example, we call the **json()** method, which deserializes the response body from a JSON string into a Javascript object.

# What are promises?

A **Promise** is an object representing the eventual completion or failure of an asynchronous operation.

We use the **then** method to register callback functions that will be triggered upon success or failure

```
const promise = doSomething();
```

```
promise.then(successCallback,  
failureCallback);
```

## Promises can be chained

The code on the right side of the screen would run in the order in which it is declared, no matter how much time each individual step takes

```
doSomething()  
  .then(result => doSomethingElse(result))  
  .then(newRes => doThirdThing(newRes))  
  .then(finalRes => console.log(`Final  
result: ${finalRes}`))  
  .catch(failureCallback);
```



## When to call fetch()

- As we saw in the previous chapter, **componentDidMount** (or **useEffect**) is the right place to call any asynchronous tasks such as HTTP requests.
- In this example, we fetch some data, get the body of the response, and assign that deserialized body to the state of our component:

```
componentDidMount() {  
  fetch('http://localhost:8000/data')  
    .then(response => response.json())  
    .then(data => this.setState({licensePlates: data }));  
}
```

## Forms

HTML form elements work a little bit differently from other DOM elements in React, because form elements naturally keep some internal state.

For example, this form accepts a single name that will be stored in the **value** property of the **input** HTML element:

```
<form>
  <label>
    Name:
    <input type="text" name="name" />
  </label>
  <input type="submit" value="Submit" />
</form>
```

## Forms

- This form has the default HTML form behavior of browsing to a new page when the user submits the form
- If you want this behavior in React, it just works
- In most cases, it's convenient to have a JavaScript function that handles the submission of the form and has access to the data that the user entered into the form
- The standard way to achieve this is with a technique called **controlled components**

## Controlled Components

In React, mutable state is kept in the state property of components, and only updated with `setState()`

- We can make the **React state** be the “**single source of truth**” for our form
- Then the React component that renders a form also controls what happens in that form on subsequent user input
- This is called a **controlled component**

# Controlled Components

In order to have a controlled component, we need two things:

- 1) The current value of the HTML form element is read from our React state
- 2) Changing the value of the form updates the React state

```
class FlavorForm extends React.Component {  
  
  state = {value: 'coconut'};  
  
  handleChange(event) {  
    this.setState({value: event.target.value});  
  }  
  
  handleSubmit(event) {  
    alert('Your favorite flavor is: ' + this.state.value);  
    event.preventDefault();  
  }  
  
  render() {  
    return (  
      <form onSubmit={(e) => this.handleSubmit(e)}>  
        <label>  
          Pick your favorite flavor:  
          <select value={this.state.value} onChange={(e) => this.handleChange(e)}>  
            <option value="grapefruit">Grapefruit</option>  
          </select>  
        </label>  
      </form>  
    );  
  }  
}
```

## Controlled Components for different HTML form elements

`<input type="text">`, `<textarea>`, and `<select>` all work very similarly - they all have a `value` attribute that you can use to implement a controlled component.

You can pass an array into the `value` attribute, allowing you to select multiple options in a `select` tag:

```
<select multiple={true} value={['B', 'C']}>
```

## Handling Multiple Form Elements

When you need to handle multiple controlled input elements, you can add a **name** attribute to each element and let the handler function choose what to do based on the value of **event.target.name**

```
class Reservation extends React.Component {  
  
  state = {  
    isGoing: true,  
    numberOfGuests: 2  
  };  
  
  handleInputChange = (event) => {  
    const target = event.target;  
    const value = target.type === 'checkbox' ? target.checked : target.value;  
    const name = target.name;  
    this.setState({[name]: value});  
  }  
  
  render() {  
    return (  
      <form>  
        <label>  
          Is going:  
          <input  
            name="isGoing" type="checkbox" checked={this.state.isGoing}  
            onChange={this.handleInputChange} />  
        </label>  
      </form>  
    );  
  }  
}
```

## Handling Multiple Form Elements

Note how we used the ES6 computed property name syntax to update the state key corresponding to the given input name:

```
this.setState({  
  [name]: value  
});
```



## React Component Router

React was designed for Single Page Applications, but very often we need more than one page!

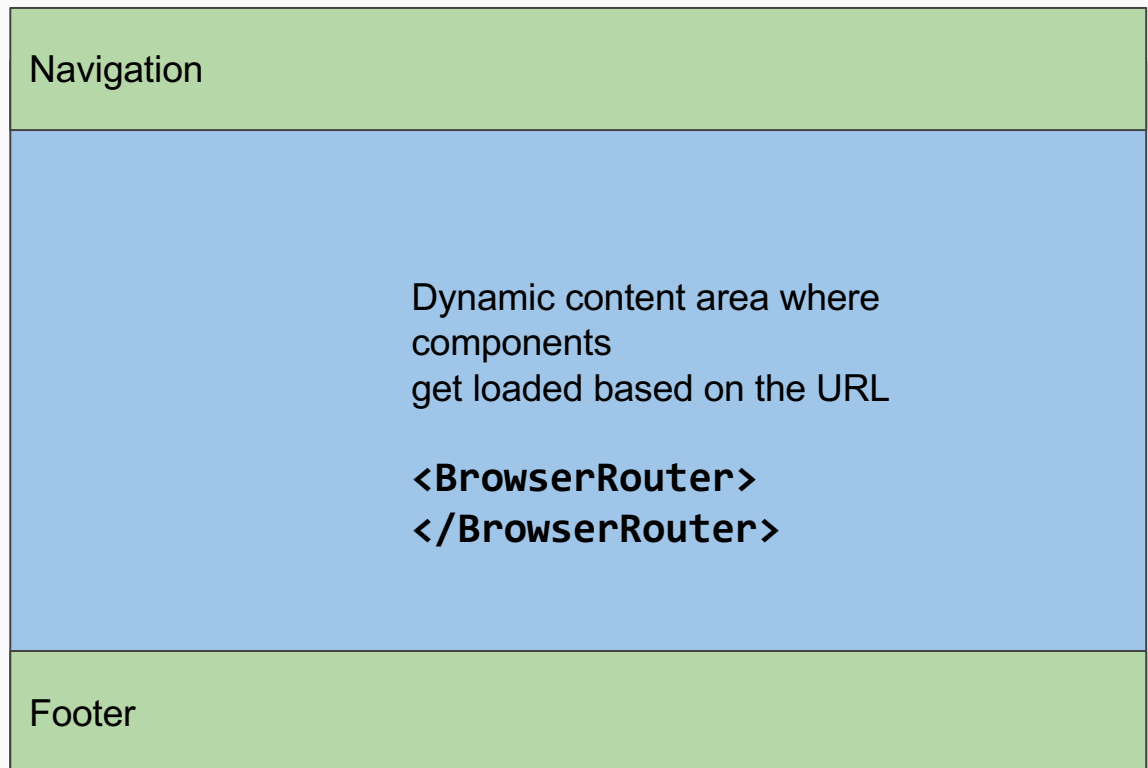
The React Router, which is a separate library (<https://reactrouter.com>), allows us to emulate this by loading components based on URLs

For instance, **/cart** would load a Cart component, **/login** for login, etc.

# Example of router

Here the navigation component and the footer are always the same on all pages.

The only dynamic part is the blue section, determined by the **BrowserRouter** component



## React Router Config

- Requires **BrowserRouter** and **Route** components that map URLs to components:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<App />} />
    <Route path="expenses" element={<Expenses />} />
    <Route path="invoices" element={<Invoices />} />
  </Routes>
</BrowserRouter>,
```

## React Router Links

Router links can be used to navigate between routes using the **Link** component and the **to** prop to set the destination:

```
<Link to="/invoices">Invoices</Link>  
<Link to="/expenses">Expenses</Link>
```